

AutoChair : Audio & Voice Subsystem PRD

Product : AutoChair – AI-Powered Smart Wheelchair

Company : Aivon Innovations Pvt. Ltd.

Subsystem : Audio & Voice Interaction

PRD Status : Development Specification

Current Stage : IN DEVELOPMENT

Primary Platform : Raspberry Pi 4 + ESP32 architecture

Purpose : Develop the audio/voice software independently while wheelchair electrical inspection continues.

1. Purpose

The Audio & Voice Subsystem will provide AutoChair with two-way audio interaction:

1. **Voice Input :** The user can speak commands through a microphone.

2. **Audio Output :** AutoChair can respond through speakers with:

- Voice feedback
- System messages
- Warning messages
- Obstacle alerts
- Confirmation messages

The subsystem should be developed independently from the wheelchair's motor-control hardware so that it can be tested before the actual wheelchair controller interface is verified.

2. Hardware Available

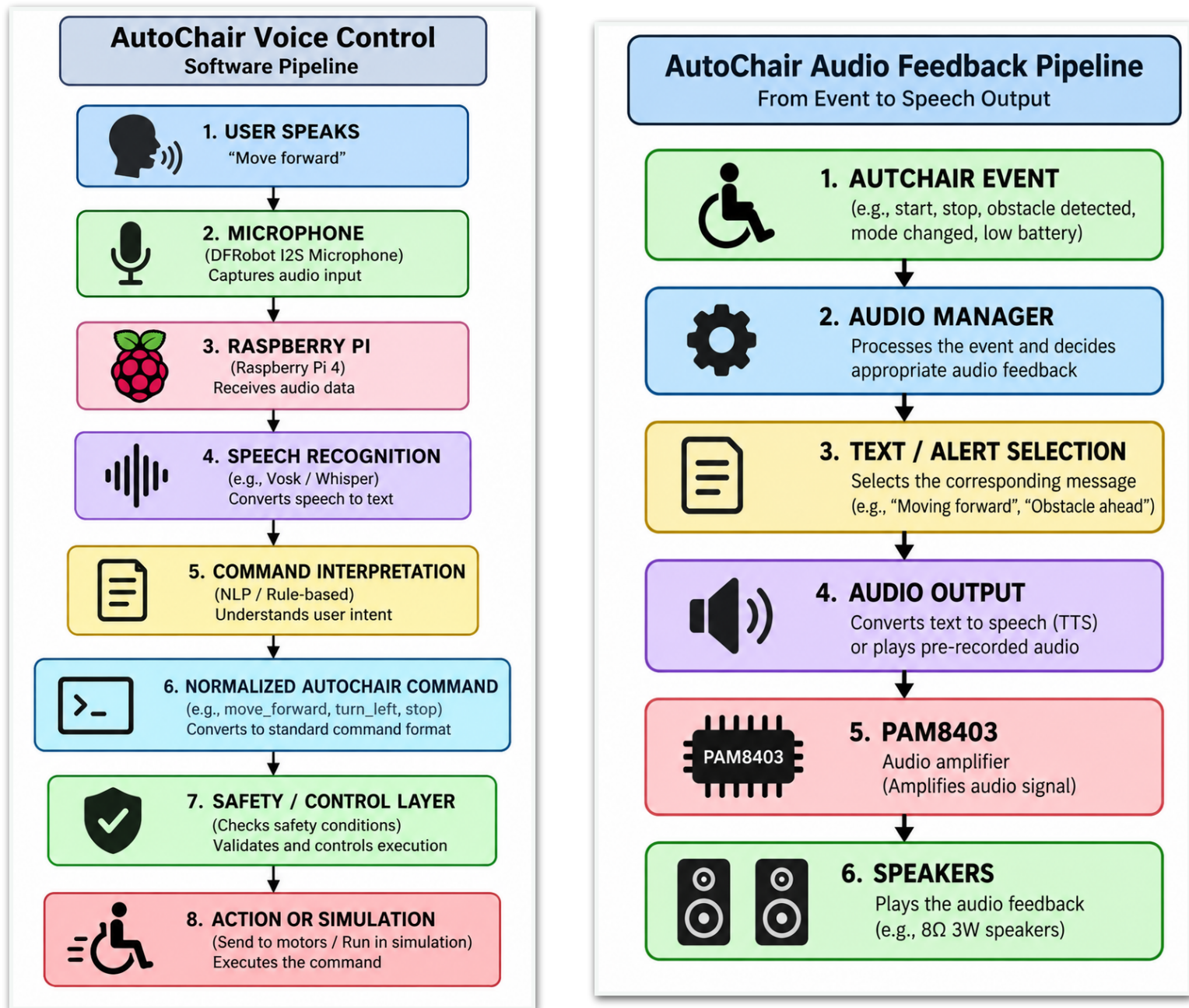
These components are already purchased and should be used before considering additional purchases.

Component	Qty	Role	Status
Raspberry Pi 4 Model B 4GB	1	Main processing	✓ Purchased
DFRobot I2S Microphone	1	Voice input	✓ Purchased
PAM8403 Audio Amplifier	1	Speaker amplification	✓ Purchased
8Ω 3W Speaker	2	Audio output	✓ Purchased
Active Buzzer Module	3	Simple warning/alert output	✓ Purchased
Waveshare 7" Touchscreen	1	Display voice status/commands	✓ Purchased

The master component list confirms the microphone, Raspberry Pi, PAM8403, speakers and buzzers are already available.

Important : Purchasing additional audio hardware is not part of the current plan unless testing identifies a genuine requirement.

3. Product Goal



4. Scope

4.1 In Scope

A. Microphone input : Capture the user's speech through the DFRobot I2S microphone.

B. Speech recognition : Convert spoken audio into text/recognized commands.

C. Command recognition : Recognize initial commands such as:

- **Start**
- **Stop**
- **Move forward**

Basic command recognition should be the first target.

D. Command normalization : Different ways of saying the same command should map to one internal command.

Example : "Stop" , "Please stop" , "Stop the chair" → STOP

E. Audio output : Play spoken/system feedback through the speakers.

F. Warning audio : Provide audible warnings for system events.

G. Obstacle audio alerts : The audio subsystem should eventually receive an obstacle event from the safety/obstacle-detection system and generate an appropriate warning.

H. Touchscreen integration : The touchscreen should be able to display:

- Listening state
- Recognized command
- Last command
- Audio/system status
- Error state

5. Out of Scope for First Version

1. ❌ Full autonomous voice navigation

- For example: "Take me to the kitchen." This is a **TARGET**, not an initial implemented capability.

2. ❌ Direct voice → motor control

- Voice commands must not directly control the wheelchair motors during initial development.

3. ❌ Automatic braking through voice software

- Braking depends on the actual wheelchair controller/brake architecture.

4. ❌ Continuous conversation/AI assistant

- We don't need a ChatGPT-like conversational system for the first prototype.

5. **✗ Wake-word system initially**

- Something like: "Hey AutoChair..." can be considered later.

6. **✗ Cloud-dependent architecture as a requirement**

- The first architecture should not depend on cloud connectivity unless we deliberately choose it later.

6. Initial Voice Command Set

Spoken Command	Internal Command	Initial Behaviour
Start	START	Software command/event
Stop	STOP	Software command/event
Move forward	MOVE_FORWARD	Software command/event

Later commands can include: MOVE_BACKWARD , TURN_LEFT , TURN_RIGHT , SLOW_DOWN , SPEED_UP but we should not implement a huge command vocabulary before the basic pipeline works.

7. Destination Commands

Potential future commands : "Go to kitchen." & "Take me to bedroom." These belong to the AutoChair's intended route-memory/navigation capability.

However: Voice recognition of the sentence $\neq$ autonomous navigation.

The system may eventually process:

```

"Go to kitchen"
  ↓
DESTINATION
  ↓
KITCHEN
  ↓
ROUTE PLANNER
  ↓
NAVIGATION

```

But that complete chain should only be considered implemented after it has actually been developed and tested. This distinction is explicitly required by the master context.

8. Audio Output Requirements

- The speaker system should support several categories.

8.1 System Messages

Examples: "AutoChair system ready." , "Voice control active." , "Voice control disabled."

8.2 Command Confirmation

Example: User: "Stop." & AutoChair: "Stopping."

8.3 Warning Messages

Example: "Obstacle detected." "Please stop."

8.4 System Errors

Example: "Microphone unavailable." "Voice command not recognized."

8.5 Navigation Feedback — Later

Potential future messages: "Destination selected." , "Route starting." , "Arriving at destination." These depend on actual navigation functionality.

9. Audio Alert Priority

Not every sound should have equal priority. We should eventually define a priority system like:

CRITICAL SAFETY



SAFETY WARNING



SYSTEM ERROR



COMMAND CONFIRMATION



NORMAL INFORMATION

For example, if the system is speaking: "System ready." and an urgent obstacle event occurs, the safety alert should take precedence.

10. Buzzer vs Speaker

We should keep these functions conceptually separate.

1. Speaker -> Used for:

- Voice feedback
- Spoken warnings
- System messages
- User interaction

2. Buzzer -> Used for:

- Immediate simple alerts
- Repetitive warning patterns
- Situations where a spoken message isn't necessary

So : SPEAKER → "Obstacle detected."

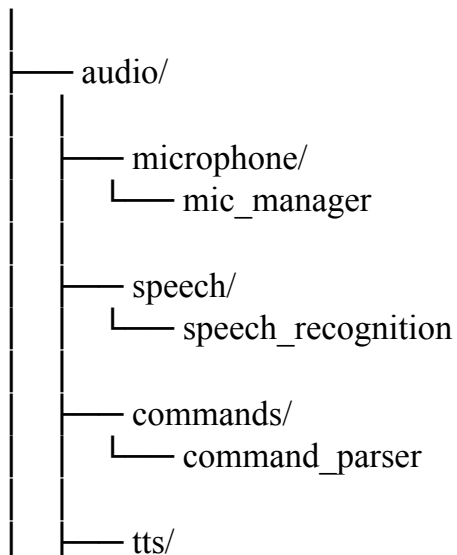
BUZZER → rapid warning pattern

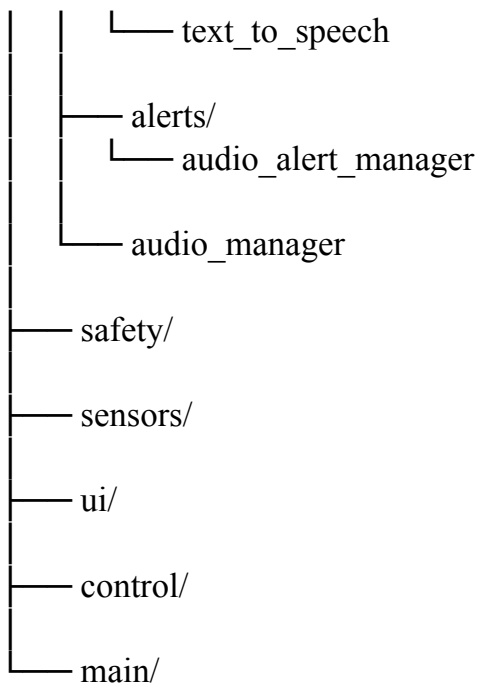
Potentially both can operate together depending on the final safety design.

The master context explicitly lists LEDs, buzzer and speaker as possible real-time feedback mechanisms.

11. Software Architecture

AUTOCHAIR SOFTWARE





This is a **proposed development architecture**, not something that should be assumed to already exist in your current code.

12. Core Software Modules

Module 1 — Microphone Manager

Responsible for:

- Initializing microphone
- Capturing audio
- Detecting microphone availability
- Handling recording
- Reporting microphone errors

Module 2 — Speech Recognition

Responsible for:

Audio

↓

Speech recognition

↓

Text

Example:

Audio input
↓
"move forward"

Module 3 — Command Parser

Responsible for:

Raw speech/text
↓
Interpretation
↓
Normalized command

Example: "please stop the chair" —> STOP

13. Command Interface

This is one of the **most important architectural decisions**.

Voice should NOT know how the motors work.

Instead:

VOICE
↓
COMMAND
↓
COMMON COMMAND INTERFACE

The same interface can later receive commands from:

Joystick ———┐
Touchscreen ———┤——→ Command Layer
Voice ———┘

This fits AutoChair's multimodal architecture of joystick + touchscreen + voice.

Therefore, if the hardware team later discovers how the wheelchair controller can safely be interfaced, we can connect the **control layer** without rewriting the voice subsystem.

14. Safety Architecture

The voice subsystem should follow:

VOICE COMMAND
↓
COMMAND PARSER
↓
SAFETY VALIDATION
↓
CONTROL SYSTEM

For example:

"STOP"
↓
STOP COMMAND
↓
SAFETY SYSTEM
↓
CONTROL ACTION

And eventually:

"Move forward"
↓
MOVE_FORWARD
↓
Safety checks
↓
Obstacle state
↓
Motion controller
↓
Verified wheelchair interface

Until that final interface is verified, the action should remain **simulated/logged**.

15. System States

The audio subsystem should have clear states.

IDLE
↓
LISTENING
↓

PROCESSING



COMMAND_RECOGNIZED



COMMAND_SENT



FEEDBACK



IDLE

Error path:

LISTENING



PROCESSING



NOT_RECOGNIZED



FEEDBACK



IDLE

Example:

LISTENING...

User speaks: "Stop"

Then: PROCESSING...

Then: COMMAND: STOP

Then: "Stopping."

The existing touchscreen dashboard can eventually display:

AUTOCHAIR		
Voice: READY		
Last Command:		

MOVE FORWARD	
Status: WAITING	
 Listening	

Important:

If the UI displays:

Speed: 3.2 km/h

that must remain a placeholder until live hardware feedback is available. The master context explicitly requires that placeholder values not be represented as live hardware data.

17. Error Handling

The subsystem should handle at least:

- **Microphone unavailable : MIC_ERROR**
- **Audio output unavailable : SPEAKER_ERROR**
- **Speech not understood : COMMAND_NOT_RECOGNIZED**
- **Unsupported command : COMMAND_NOT_SUPPORTED**
- **Processing timeout : VOICE_TIMEOUT**
- **Audio device initialization failure : AUDIO_INIT_ERROR**

Errors should be logged and shown appropriately on the touchscreen.

18. Logging

Every recognized command should be logged during development.

Example:

```
[20:31:04] Voice input received
[20:31:05] Speech: "Please stop"
[20:31:05] Command: STOP
[20:31:05] Command status: ACCEPTED
```

For an unrecognized command:

[20:32:10] Speech: "..."
[20:32:10] Command: UNKNOWN
[20:32:10] Status: NOT_RECOGNIZED

This will be extremely useful when debugging with Antigravity.

19. Testing Strategy

We should test in stages.

Test 1 — Speaker

Raspberry Pi



PAM8403



Speaker

Expected: Clear audio output.

Test 2 — Microphone

Microphone



Pi

Expected: Pi receives audio successfully.

Test 3 — Speech recognition

Say: "Stop"

Expected: STOP

Test 4 — Multiple phrases

Test: Stop , Please stop , Stop the chair

Expected:

STOP

STOP

STOP

Test 5 — Unknown speech

Say something unrelated.

Expected: UNKNOWN , No movement.

Test 6 — Audio feedback

Recognized command: STOP

Expected: 🗣️ appropriate confirmation.

Test 7 — Touchscreen

Verify that the recognized command appears on the UI.

Test 8 — Simulated control

Voice: "Move forward"

Software: MOVE_FORWARD

But **do not physically move the wheelchair yet.**

20. Development Milestones

M0 — Architecture

Goal: Define subsystem and interfaces.

Status: 🟡 IN DEVELOPMENT

M1 — Audio Output

Goal: Pi → amplifier → speakers

Status: 🟡 To develop/test

M2 — Microphone

Goal: Microphone → Pi

Status: 🟡 To develop/test

M3 — Speech Recognition

Goal: Speech → text

Status: 🟡 To develop/test

M4 — Command Parser

Goal: Text → normalized AutoChair command

Status: 🟡 To develop/test

M5 — Audio Feedback

Goal: System event → spoken response

Status: 🟡 To develop/test

M6 — Touchscreen Integration

Goal: Voice state + command displayed on UI.

Status: 🟡 To develop/test

M7 — Safety/Event Integration

Goal: Obstacle/system events → audio alerts.

Status: 🟡 IN DEVELOPMENT

M8 — Hardware Integration

Goal: Connect command layer to the **verified wheelchair control interface**.

Status: 🕒 BLOCKED until hardware interface is safely verified.

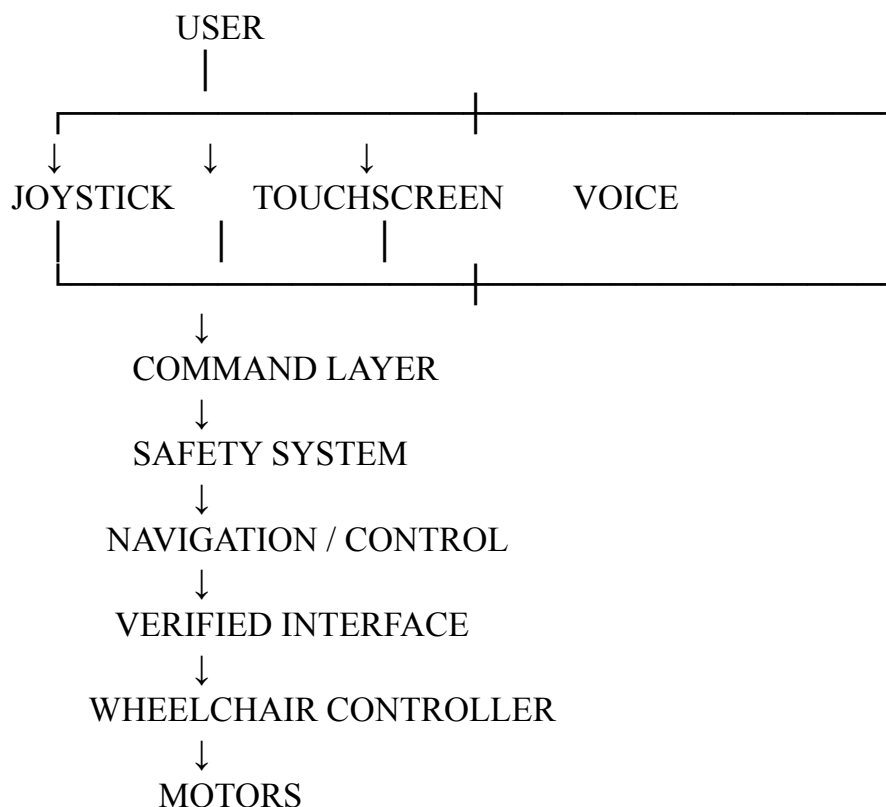
21. Definition of Done — First Audio Prototype

We can call the **Audio/Voice Prototype** successful when:

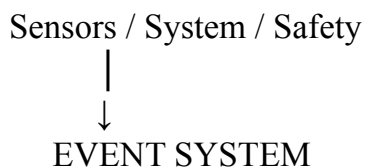
- Raspberry Pi detects microphone
- Raspberry Pi captures audio
- Raspberry Pi outputs audio
- PAM8403 drives speakers correctly
- Speech recognition works
- Start recognized
- Stop recognized
- Move forward recognized
- Unknown speech handled safely
- Commands normalized
- Commands logged
- Audio feedback works
- Touchscreen displays voice state
- No voice command directly drives wheelchair motors
- Software can operate without the wheelchair's motor controller
- Hardware-control interface remains modular

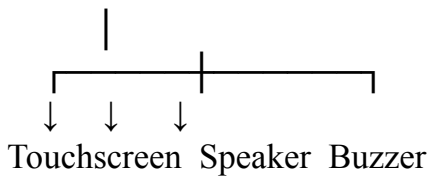
22. Final Architecture Goal

Ultimately, Audio should become one module inside the larger AutoChair architecture:



And the reverse feedback path:





That gives us the modularity we want.

23. Development Rule

The most important rule for Antigravity and our future code:

The Audio & Voice subsystem must not contain wheelchair-specific motor-control implementation.

It should generate **commands/events**, not directly manipulate motors.

That way:

Today VOICE → COMMAND → SIMULATION

and later: VOICE → COMMAND → SAFETY → VERIFIED CONTROLLER → WHEELCHAIR

without rebuilding the voice system.

This is exactly how we can keep making software progress **while the physical wheelchair inspection is still underway**. The master context itself identifies microphone/voice processing, touchscreen, sensor acquisition, safety logic and controller integration as separate development areas.